

UNIT 5

Introduction to I/O Management

Input/Output (I/O) Management deals with how the **Operating System (OS)** handles the *communication between the computer system and external devices* such as disks, printers, keyboards, and networks.

In a computer system, **Input/Output (I/O) Organization** refers to how the **operating system and hardware** coordinate to perform **data transfer** between **I/O devices** (like keyboard, disk, printer) and the **main memory/CPU**.

Since **I/O devices** are much **slower** than the CPU, proper organization is essential for:

- Efficient data transfer
- Reducing CPU idle time
- Handling multiple devices concurrently

I/O System Components

An I/O system typically consists of:

1. **I/O Devices** – hardware components used for input/output (e.g., disk, mouse, printer).
2. **Device Controller** – an electronic component that operates a particular type of device (keyboard controller, disk controller, etc.).
 - A **hardware interface** between the CPU and the I/O device.
 - Each type of device has its own controller (e.g., disk controller, printer controller).
 - It contains:

Data Register – stores data to transfer.

Status Register – indicates device state (busy, ready, error).

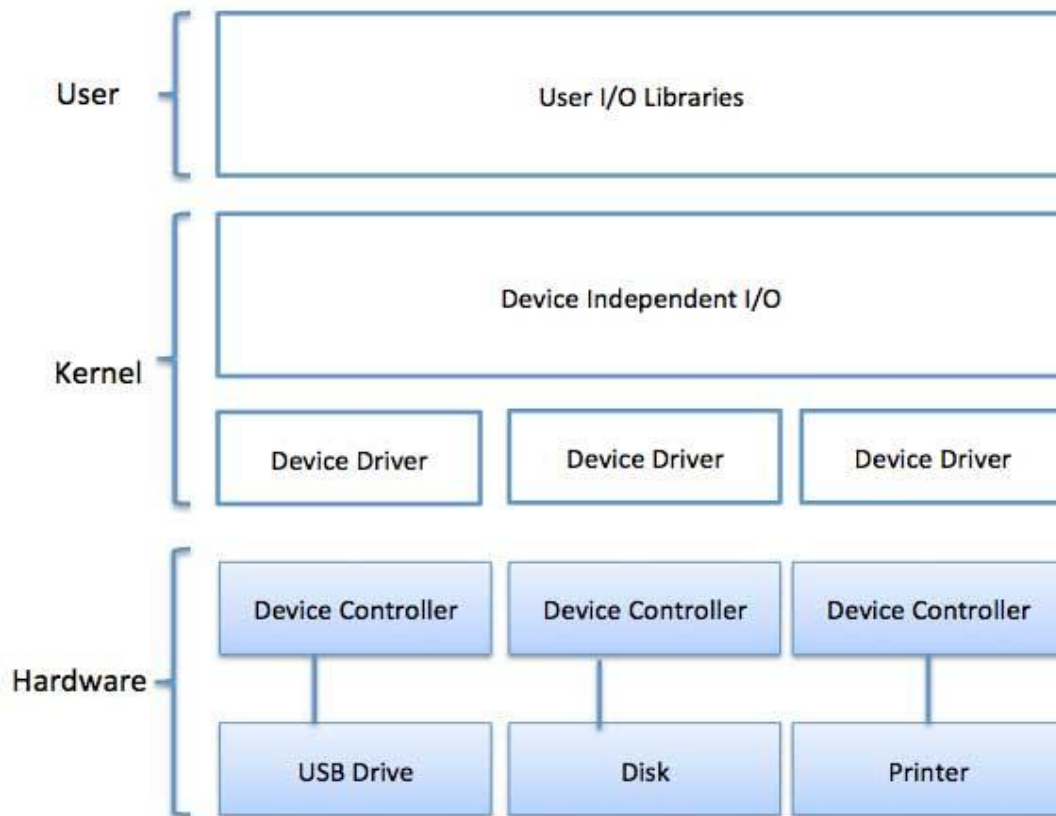
Control Register – receives commands from CPU.

3. **Device Driver** – software that provides an interface between the OS and the device controller. It provides a **uniform interface** to the OS, hiding hardware details. Each device has its own driver.
4. **Interrupts** – signals sent to the CPU to indicate an I/O operation has completed.
5. **DMA (Direct Memory Access)** – allows devices to transfer data directly to/from memory without CPU involvement

Layers of I/O System Organization

I/O in OS is organized into **multiple layers**, each with a specific role:

Layer	Function
User-Level I/O	Application program uses system calls (e.g., fopen, printf, scanf).
System Call Interface	Provides standardized interface between user program and OS.
Device-Independent I/O Software	Handles buffering, caching, spooling, and naming of devices.
Device Drivers	Translates OS requests into device-specific commands.
Interrupt Handlers	Handle device interrupts and signal OS when I/O completes.
Hardware (Device Controllers)	Actually perform the data transfer between device and memory.

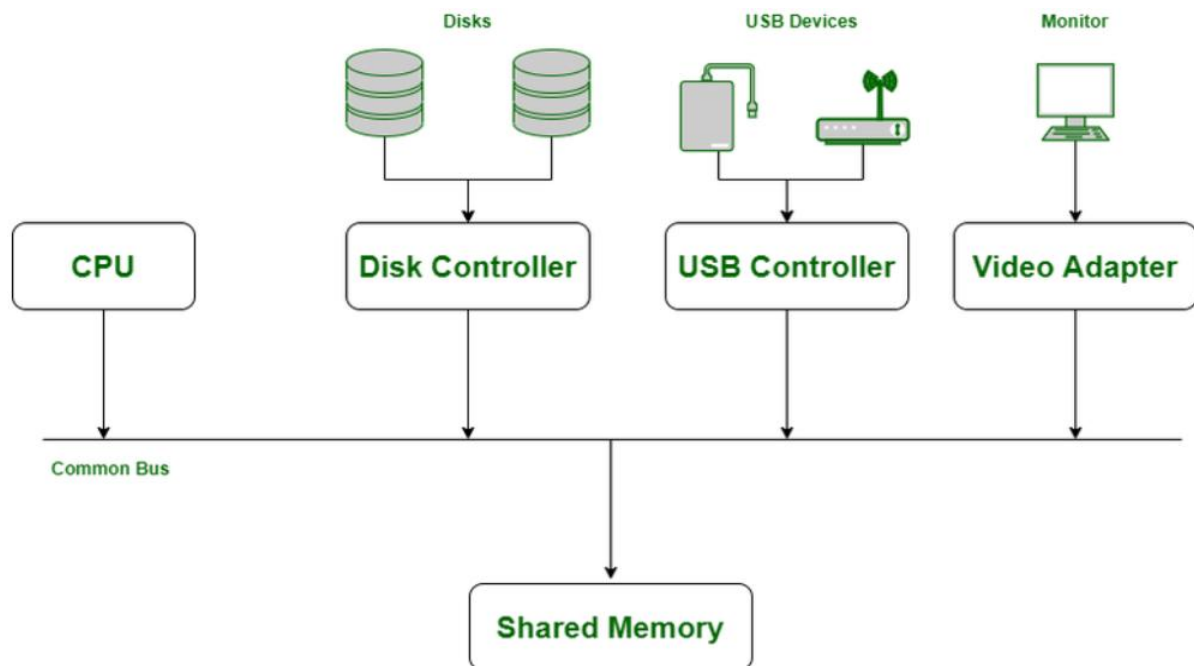


How I/O Operations Are Performed

I/O operations involve communication between the CPU and external devices like hard drives, keyboards, and printers. These operations are managed by the operating system, device drivers, and other system programs. Following are the steps involved in I/O Operations:

- 1. I/O Request Initiation:** When a user or program requests an I/O operation (like opening a file), the OS communicates with the device driver to handle the request.
- 2. I/O Traffic Controller:** The I/O Traffic Controller keeps track of the status of all devices, control units, and communication channels. It ensures that devices are ready and available to handle the request.
- 3. I/O Scheduler:** The I/O Scheduler determines the order in which I/O requests are processed. It manages access to devices based on priority, fairness, and availability to optimize system performance.
- 4. I/O Device Handler:** The I/O Device Handler manages device interrupts and oversees the data transfer. It ensures that data is transferred between the device and memory (or CPU) properly.

5. Completion and Notification: Once the I/O operation is complete, the OS informs the program or user that the task is finished.



Methods of I/O Data Transfer

There are several ways to organize the transfer of data between CPU and I/O devices:

A. Programmed I/O (Polling Method)

- CPU **directly controls** all I/O operations.
- CPU keeps **checking the device status** until it is ready.
- Once ready, CPU sends/receives data.

Process:

CPU -> Check status -> Wait until ready -> Transfer data

Advantages:

- Simple to implement.

Disadvantages:

- CPU remains **busy waiting**, wasting time.
- Inefficient for slow devices.

Example: Reading keyboard input using a loop.

B. Interrupt-Driven I/O

- Device **sends an interrupt** to CPU when it is ready.
- CPU does not need to continuously check device.
- When interrupt occurs → CPU **pauses current work**, executes **Interrupt Service Routine (ISR)** to handle I/O.

Process:

CPU -> Executes program

-> Device sends interrupt when ready

-> CPU handles interrupt (via ISR)

-> CPU resumes program

Advantages:

- Efficient CPU usage.
- Faster response time.

Disadvantages:

- Handling multiple interrupts can be complex.

C. Direct Memory Access (DMA)

- **DMA Controller (DMAC)** handles data transfer **directly between memory and I/O device**, bypassing CPU for most operations.

Process:

1. CPU initializes DMA (source, destination, size).
2. DMA takes control of system bus.
3. DMA transfers data between memory and device.
4. DMA notifies CPU after completion (interrupt).

Advantages:

- High-speed transfer.
- Frees CPU for other tasks.

Used in: Disk drives, network cards, video streaming.

I/O Buffering

I/O devices are slower than CPU and memory. Buffering smooths speed mismatch by temporarily storing data.

1. Single Buffering

- Only **one buffer** is used.
- While the **I/O device fills the buffer**, the process **waits**.
- After buffer is full → process reads data → device fills buffer again.

Advantages:

- Better than no buffering.
- Simple to implement.

Disadvantages:

- CPU still waits when buffer is being filled/refilled.
- Limited overlap of I/O and computation.

2. Double Buffering (Twin Buffering)

- Uses **two buffers** alternately.
- While one buffer is being **filled** by I/O device, the other buffer is being **used** by the process.

Example:

- While process is reading from **Buffer 1**, I/O device writes new data to **Buffer 2**.
- When both are done, buffers are swapped.

Advantages:

- CPU and I/O can work in parallel.
- Better performance and throughput.

Disadvantages:

- Needs more memory.
- Slightly more complex management.

3. Circular (Multiple) Buffering

- More than two buffers used in a **circular queue**.
- Very useful when data flow is **continuous** (e.g., streaming, video, real-time systems).

Advantages:

- Maximum overlap of I/O and computation.
- Suitable for high-speed or continuous data transfer.

Disadvantages:

- Increased complexity in buffer management.
- Higher memory usage.

Comparison

Type	Description	Example
Single Buffering	One buffer in main memory; CPU waits when buffer is empty/full.	Input from keyboard
Double Buffering	Two buffers used alternately; one fills while other empties.	Reading a file from disk while processing previous block
Circular Buffering	Multiple buffers arranged in circular order for continuous data flow.	Video/audio streaming

Advantages:

- Improves CPU utilization.

- Allows overlap of I/O and computation.

Disk I/O

Structure of a Disk

- **Platters:** Circular disks coated with magnetic material.
- **Tracks:** Concentric circles on each platter.
- **Sectors:** Subdivision of a track.
- **Cylinder:** Set of tracks vertically aligned across platters.

Disk Access Time

Total time to access data on disk =

Access Time = Seek Time + Rotational Latency + Transfer Time

- **Seek Time:** Time to move head to the required track.
- **Rotational Latency:** Time waiting for desired sector to rotate under the head.
- **Transfer Time:** Time to read/write data once sector is located.

Operating System Design Issues for I/O

1. **Device Independence:** Same I/O interface for different devices.
2. **Error Handling:** Automatic detection and recovery from I/O errors.
3. **Concurrency:** Managing multiple devices and processes simultaneously.
4. **Security & Protection:** Restricting unauthorized access to I/O devices.
5. **Spooling:** Overlapping of output of one job with input of another (used for printers).
6. **Caching:** Storing frequently accessed data in fast memory for quick retrieval.

7. **Buffer Management:** Allocation and deallocation of I/O buffers efficiently.

File Concept

A *file* is a named collection of related information that is recorded on secondary storage (like a disk).

It is the smallest unit of logical storage from the user's perspective.

File Attributes

Each file is described by a set of attributes stored in the directory entry. Common attributes:

- **Name:** Human-readable identifier.
- **Identifier:** Unique tag (number) to locate file within the file system.
- **Type:** Indicates type (text, binary, executable, etc.)
- **Location:** Pointers to file location on disk.
- **Size:** Current file size (in bytes, words, etc.)
- **Protection:** Access control information.
- **Time, Date, and User ID:** For creation, modification, and access.

File Operations

Typical operations provided by the OS:

1. **Create:** Make a new file entry in directory.
2. **Write:** Store data into file.
3. **Read:** Retrieve data from file.
4. **Reposition (Seek):** Move file pointer to a specific location.
5. **Delete:** Remove file from directory.
6. **Truncate:** Erase file content without deleting file itself.
7. **Open/Close:** Prepare file for access / release control when done.

OS maintains a table of *open files* for each process to manage open/close operations efficiently.

File Types

- **Text files:** Sequence of characters.
- **Source files:** Human-readable program code.
- **Binary files:** Compiled executables, images, videos.
- **System files:** Device drivers, configuration data, etc.

Directory and Disk Structure

Directory Overview

A *directory* is a collection of entries that contain information about all files in a file system.

Each entry stores:

- File name
- File type
- File location
- Size
- Access rights
- Timestamps

Directory Operations

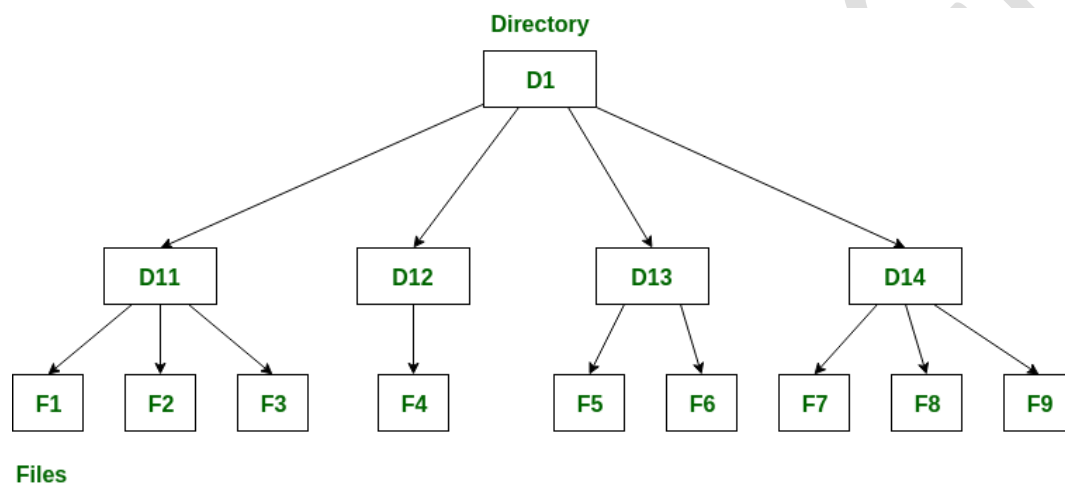
1. Search for a file
2. Create a file
3. Delete a file
4. List directory contents
5. Rename a file
6. Traverse directory structure

Structures of Directory in Operating System

The operating system uses directories to track where files are stored, just like using folders to organise papers.

- Different directory structures can be used to suit various organisational needs.
- Understanding directory structures helps in organising and accessing files more easily.

By using these structures, it becomes simpler to manage and navigate files on your computer.



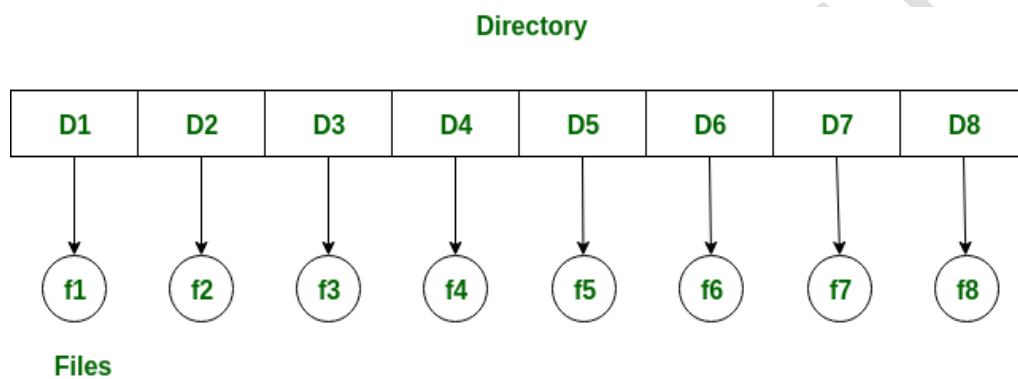
Different Types of Directories in OS

In an operating system, there are different types of directory structures that help organise and manage files efficiently. Each type of directory has its own way of arranging files and directories, offering unique benefits and features. These are

- Single-Level Directory
- Two-Level Directory
- Tree Structure/ Hierarchical Structure
- Acyclic Graph Structure
- General-Graph Directory Structure

1) Single-Level Directory

- The single-level directory is the **simplest directory structure**.
- In it, all files are contained in the same directory which makes it easy to support and understand.
- A single level directory has a significant limitation, however, when the number of files increases or when the system has more than one user.
- Since all the files are in the same directory, they must have a **unique name**. If two users call their dataset test, then the unique name rule is violated.



- Since it is a single directory, so its implementation is very easy.
- If the files are smaller in size, searching will become faster.
- The operations like file creation, searching, deletion, updating are very easy in such a directory structure.

Advantages

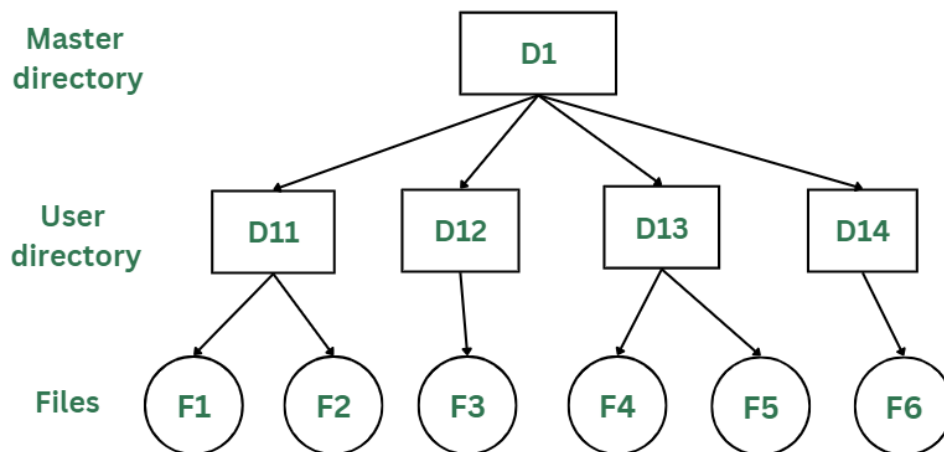
- **Logical Organization:** Arrange files hierarchically for easy navigation.
- **Efficiency:** Faster file searching and access.
- **Security:** Restrict access at directory level to protect data.
- **Backup & Recovery:** Simplifies locating and restoring files.
- **Scalability:** Easily supports growth with new files and directories

Disadvantages

- There may be chance of name collision because two files can have the same name.
- Searching will become time taking if the directory is large.
- This can not group the same type of files together.

2) Two-Level Directory

- In a two-level directory structure, each user has a separate User File Directory (UFD) containing only their files.
- A Master File Directory (MFD) stores entries for all users and points to their respective UFDs, preventing filename conflicts between users.



Two-Levels Directory Structure

Advantages

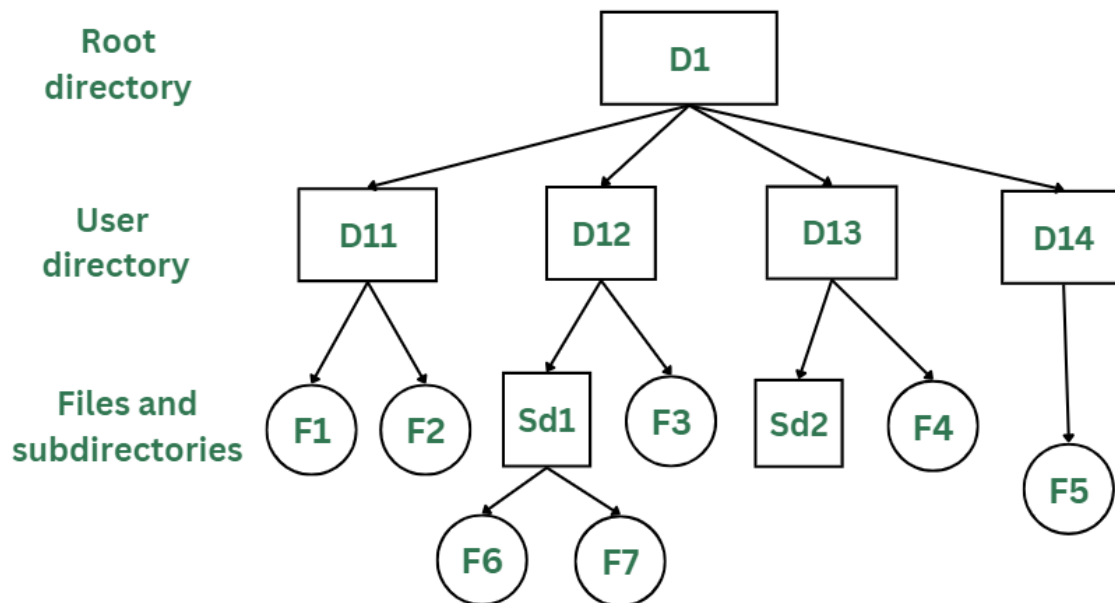
- The main advantage is there can be **more than two files with same name**, and would be very helpful if there are multiple users.
- A **security would be there** which would prevent user to access other user's files.
- Searching of the files becomes very easy in this directory structure.

Disadvantages

- As there is advantage of security, there is also disadvantage that the user **cannot share the file with the other users**.
- Unlike the advantage users can create their own files, users **don't have the ability to create subdirectories**.
- Scalability is not possible because one user can't group the same types of files together.

3) Tree Structure/ Hierarchical Structure

- The tree directory structure is the most common in personal computers. It resembles an upside-down tree, with the root directory at the top containing all user directories.
- **Each user can create files and subdirectories within their own directory** but cannot access or modify the root or other users' directories.



Tree/Hierarchical Directory Structure

Advantages

- This directory structure allows subdirectories inside a directory.
- The searching is easier.
- File sorting of important and unimportant becomes easier.
- This directory is more scalable than the other two directory structures explained.

Disadvantages

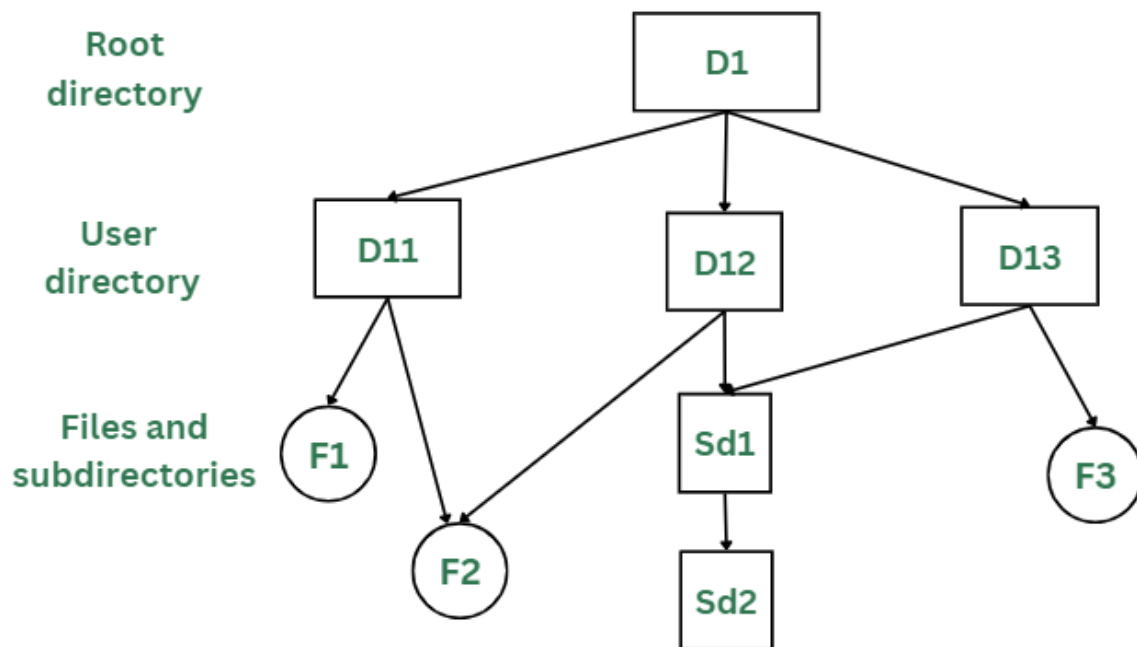
- As the user isn't allowed to access other user's directory, this prevents the file sharing among users.
- As the user has the capability to make subdirectories, if the number of subdirectories increase the searching may become complicated.
- Users cannot modify the root directory data.

- If files do not fit in one, they might have to be fit into other directories.

4) Acyclic Graph Structure

- In the earlier directory structures, a file could only be accessed from the directory it was stored in.
- The acyclic graph directory structure solves this by allowing a file or subdirectory to be shared across multiple directories using links.
- Changes made by one user are visible to all users sharing that file.

In the below figure, this explanation can be nicely observed, where a file is shared between multiple users. If any user makes a change, it would be reflected to both the users.



Acyclic Graph Structure

Advantages

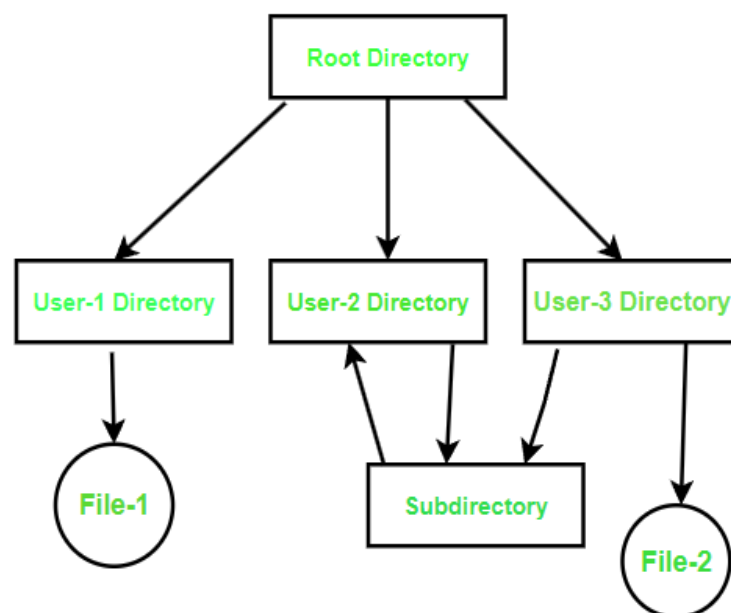
- Sharing of files and directories is allowed between multiple users.
- Searching becomes too easy.
- Flexibility is increased as file sharing and editing access is there for multiple users.

Disadvantages

- Because of the complex structure it has, it is difficult to implement this directory structure.
- The user must be very cautious to edit or even deletion of file as the file is accessed by multiple users.
- If we need to delete the file, then we need to delete all the references of the file in order to delete it permanently.

5) General-Graph Directory Structure

Unlike the acyclic-graph directory, which avoids loops, the general-graph directory can have cycles, meaning a directory can contain paths that loop back to the starting point. This can make navigating and managing files more complex.



General Graph Directory Structure

In the above image, you can see that a cycle is formed in the User 2 directory. While this structure offers more flexibility, it is also more complicated to implement.

Advantages of General-Graph Directory

- More flexible than other directory structures.
- Allows cycles, meaning directories can loop back to each other.

Disadvantages of General-Graph Directory

- More expensive to implement compared to other solutions.
- Requires garbage collection to manage and clean up unused files and directories.

File Allocation Methods

The allocation methods define how the files are stored in the disk blocks. There are three main disk space or file allocation methods.

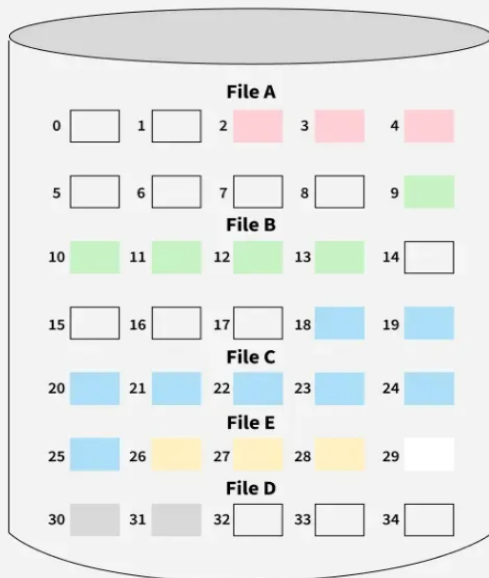
- Contiguous Allocation
- Linked Allocation
- Indexed Allocation

1. Contiguous Allocation

- In this scheme, each file **occupies a contiguous set of blocks on the disk.**
- This means that given **the starting block address and the length of the file** (in terms of blocks required), we can determine the blocks occupied by the file.
- The **directory entry for a file** with contiguous allocation contains:
 - Address of starting block
 - Length of the allocated portion.
- **Example:** If a file requires n blocks and is given a block b as the starting location, then the blocks assigned to the file will be:

$b, b+1, b+2, \dots, b+n-1.$

Continuous Allocation



File Name	Star block	Length
File A	2	3
File B	9	5
File C	18	8
File D	30	2
File E	26	3

Contiguous Allocation

Advantages

- Both the Sequential and Direct Accesses are supported by this. For direct access, the address of the k th block of the file which starts at block b can easily be obtained as $(b + k)$.
- This is extremely fast since the number of seeks are minimal because of contiguous allocation of file blocks.

Disadvantages

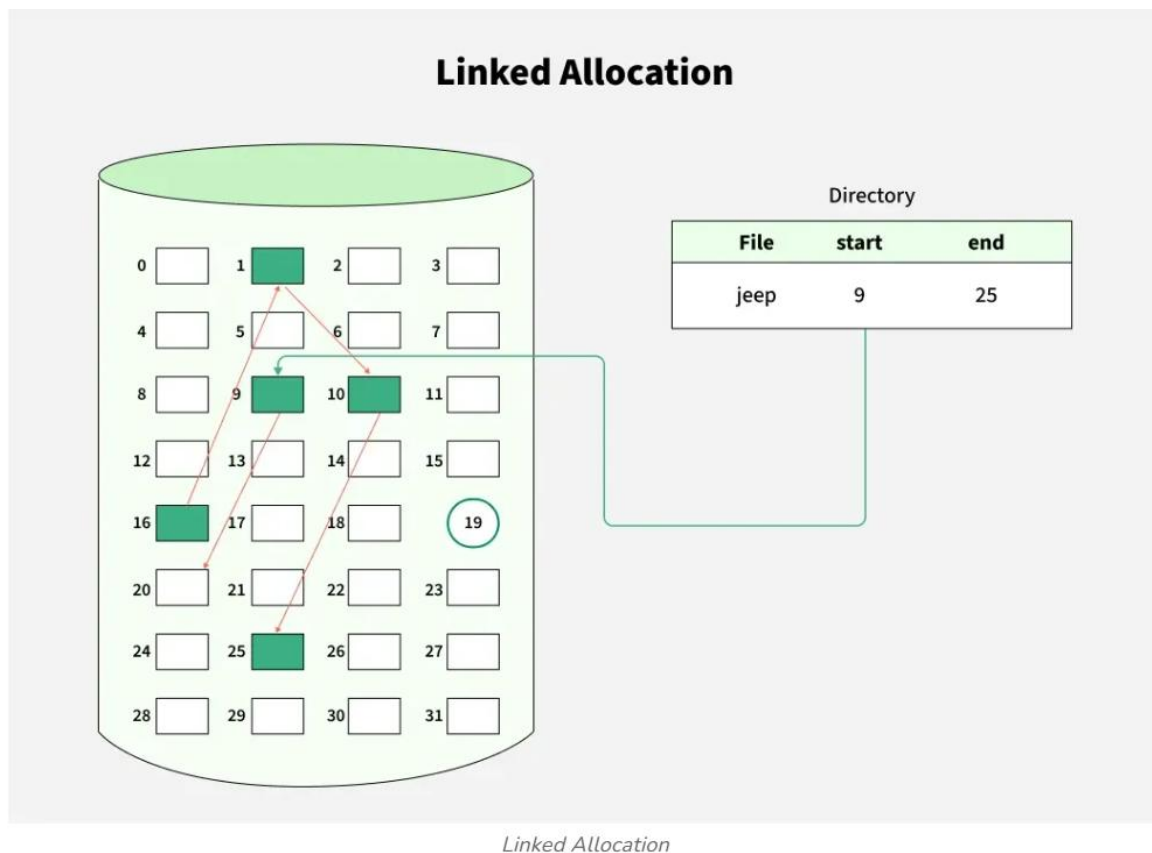
- This method suffers from both internal and external fragmentation. This makes it inefficient in terms of memory utilization.
- Increasing file size is difficult because it depends on the availability of contiguous memory at a particular instance.

2. Linked Allocation

In this scheme, each file is a **linked list of disk blocks** which need not be contiguous. Here,

- The **disk blocks can be scattered** anywhere on the disk. The directory entry contains a pointer to the starting and the ending file block.
- Each block contains a pointer to the next block** occupied by the file.

- **Example:** The file 'jeep' in following image shows how the blocks are randomly distributed. The last block (25) contains -1 indicating a null pointer and does not point to any other block.



Advantages

- This is **very flexible in terms of file size**. File size can be increased easily since the system does not have to look for a contiguous chunk of memory.
- This method **does not suffer from external fragmentation**. This makes it relatively better in terms of memory utilization.

Disadvantages

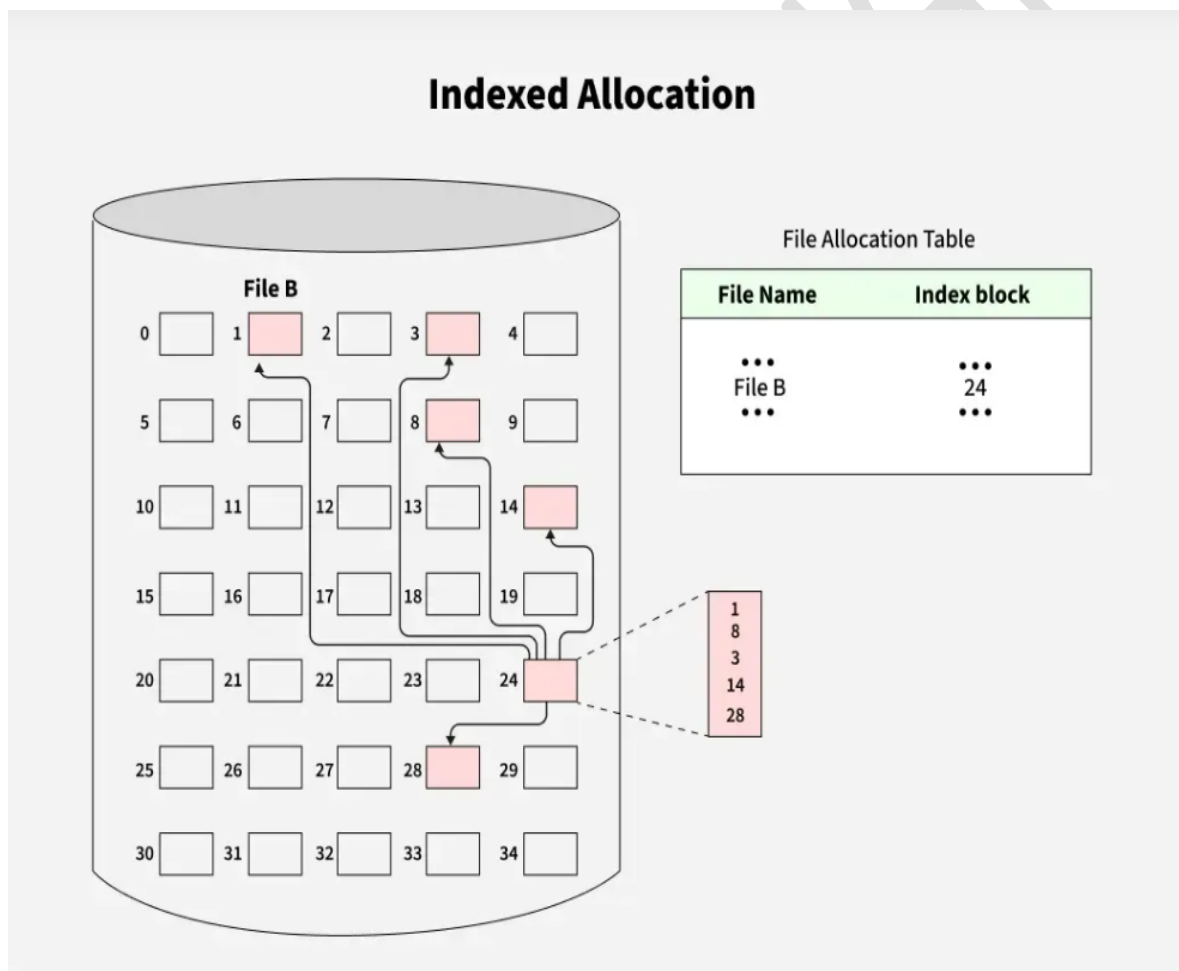
- Because the file blocks are distributed randomly on the disk, **a large number of seeks are needed to access every block individually**. This makes linked allocation slower.
- It **does not support random or direct access**. We can not directly access the blocks of a file. A block k of a file can be accessed by traversing k blocks sequentially (sequential access) from the starting block of the file via block pointers.

- **Pointers** required in the linked allocation incur **some extra overhead**.

3. Indexed Allocation

In this scheme, a special block known as the Index block contains the pointers to all the blocks occupied by a file. Here,

- **Each file has its own index block.**
- The *i*th entry in the index block contains the disk address of the *i*th file block.
- The directory entry contains the address of the index block as shown in the image.



Indexed Allocation

Advantages

- This **supports direct access** to the blocks occupied by the file and therefore provides fast access to the file blocks.

- It overcomes the problem of external fragmentation.

Disadvantages

- The pointer overhead for indexed allocation is greater than linked allocation.
- For very small files, say files that expand only 2-3 blocks, the indexed allocation would keep one entire block (index block) for the pointers which is inefficient in terms of memory utilization. However, in linked allocation we lose the space of only 1 pointer per block.

Indexing Schemes for Large Files

When a file is very large, a single index block may not have enough space to hold all the pointers to the data blocks. To manage this, the following indexing mechanisms are used:

1. Linked Scheme

- In this approach, *multiple index blocks are linked together*.
- Each index block contains some pointers to disk blocks and a pointer to the next index block.
- The chain continues until all the pointers are covered.

2. Multilevel Index

- Instead of a single large index block, we use a *hierarchy of index blocks*.
- The first-level index block points to second-level index blocks and those second-level blocks contain pointers to the actual data blocks.
- Can be extended to three or more levels as required by file size.

3. Combined Scheme (Inode Structure)

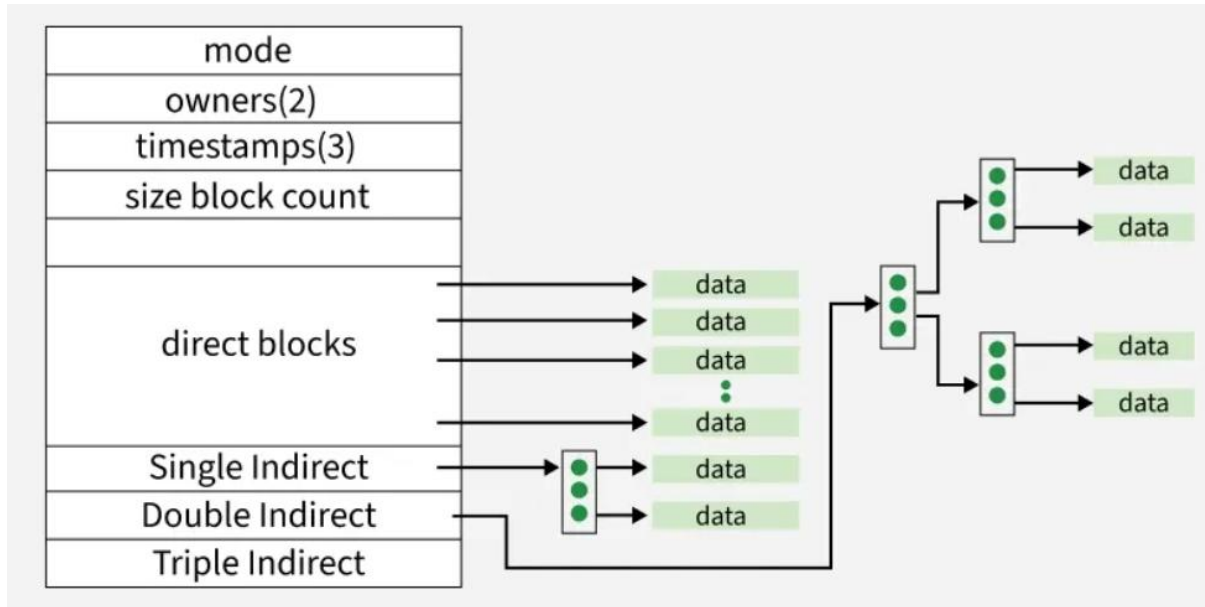
A special block called the *Inode (Information Node)* stores File metadata (name, size, permissions, timestamps) & Block addresses (pointers).

The Inode typically contains:

1. **Direct Pointers:** Point directly to data blocks (e.g., first 10 pointers).
2. **Single Indirect Pointer:** Points to a block that holds further block addresses.

3. Double Indirect Pointer: Points to a block that contains pointers to other indirect blocks, which in turn point to data blocks.

4. Triple Indirect Pointer: Adds another layer of indirection for extremely large files.



Combined Scheme (Inode Structure)

Why Combined Scheme Is Preferred:

- Combines direct access for small files (faster).
- Allows efficient scalability for large files.
- Widely used in modern filesystems like ext2, ext3, ext4 (Linux).

File System

A **File System** is the **method and data structure** that an operating system uses to **organize, store, retrieve, and manage files** on a storage device (like a hard drive, SSD, or USB).

It decides:

- How files are **named**,
- How data is **stored**,
- How **directories** and **paths** are managed, and
- How **permissions** and **metadata** are handled.

Types of File Systems

1. Disk-based File Systems

These are the most common file systems, used for **hard drives and SSDs**.

(a) FAT (File Allocation Table)

- **Developed by:** Microsoft (for MS-DOS, Windows).
- **Types:** FAT12, FAT16, FAT32.
- **Structure:** Uses a **table** to track file locations on disk blocks.
- **Advantages:**
 - Simple and fast.
 - Compatible with many OS (Windows, Linux, etc.).
- **Limitations:**
 - **File size limit:** 4 GB (for FAT32).
 - **Partition size limit:** 2 TB (FAT32).
 - No built-in file permissions or journaling.
- **Used in:** Pen drives, memory cards.

(b) NTFS (New Technology File System)

- **Developed by:** Microsoft (for Windows NT onwards).
- **Features:**
 - **Security permissions (ACLs)** – Access control lists.
 - **Compression, encryption, journaling.**
 - Supports **large files** and **disk quotas**.
- **Advantages:**
 - Reliable (journaling reduces corruption).
 - Handles large disks and files.

- **Used in:** Modern Windows systems (Windows XP → Windows 11).

(c) exFAT (Extended FAT)

- **Hybrid of FAT and NTFS.**
- Designed for **external drives** and **cross-platform use**.
- **Advantages:**
 - Supports large files (no 4 GB limit).
 - Compatible with both Windows and macOS.
- **Used in:** USBs, SD cards, external HDDs.

(d) HFS / HFS+ (Hierarchical File System)

- **Developed by:** Apple.
- **HFS:** Original Mac file system (1985).
- **HFS+:** Improved version with journaling (used in macOS until 2017).
- **Features:** Supports long filenames, file attributes, and journaling.

(e) APFS (Apple File System)

- **Introduced by:** Apple (macOS High Sierra, iOS 10.3+).
- **Features:**
 - Optimized for SSDs.
 - **Encryption, snapshots, cloning, and crash protection.**
 - Very fast and efficient storage handling.

(f) EXT Family (Extended File System)

- **Developed for:** Linux.

Version	Year	Features
EXT2	1993	No journaling, simple and efficient.
EXT3	2001	Added journaling → improves recovery.
EXT4	2008	Large file & volume support, backward compatible, better performance.

- **Used in:** Most modern Linux distributions.

(g) XFS

- **High-performance journaling file system** by Silicon Graphics (SGI).
- Supports large files and parallel I/O.
- **Used in:** Enterprise-level Linux servers.

(h) Btrfs (B-tree File System)

- **Modern Linux file system.**
- Supports **snapshotting, cloning, RAID, compression.**
- Designed for **fault tolerance and scalability.**

2. Network File Systems (NFS)

Used to **access files over a network** as if they were on a local disk.

Examples:

- **NFS (Network File System):** Developed by Sun Microsystems; used in Unix/Linux.
- **SMB/CIFS (Server Message Block):** Used in Windows for shared drives.
- **AFS (Andrew File System):** Used in distributed systems.
- **DFS (Distributed File System):** Used in Windows Server for file replication.

3. Distributed / Cloud File Systems

Used in large-scale or cloud storage environments.

Examples:

- **Google File System (GFS):** Used in Google infrastructure.
- **Hadoop Distributed File System (HDFS):** Used for Big Data applications.
- **CephFS, GlusterFS:** Scalable and fault-tolerant open-source systems.

4. Flash / Embedded File Systems

Designed for flash memory and embedded devices.

Examples:

- **JFFS2 (Journaling Flash File System 2)**
- **YAFFS (Yet Another Flash File System)**
- **F2FS (Flash-Friendly File System)** – Optimized for SSDs.

5. Special Purpose File Systems

Used for specific functions in OS.

File System	Purpose
ProcFS (/proc)	Provides process and kernel information (Linux virtual FS).
SysFS (/sys)	Exposes device and driver information.
TMPFS	Stores temporary files in RAM (faster but volatile).

Disk Scheduling (Important)

Disk scheduling algorithms manage how data is read from and written to a computer's hard disk. These algorithms help determine the order in which disk read and write requests are processed.

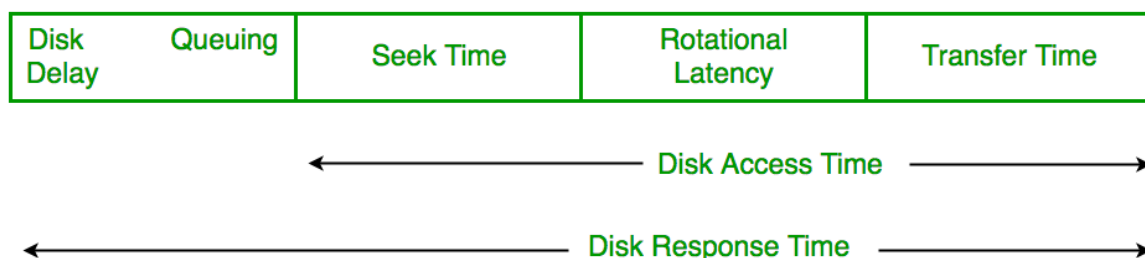
- Disk scheduling is also known as I/O Scheduling.
- The main goals of disk scheduling are to optimize the performance of disk operations, reduce the time it takes to access data and improve overall system efficiency.
- Common disk scheduling methods include First-Come, First-Served (FCFS), Shortest Seek Time First (SSTF), SCAN, C-SCAN, LOOK, and C-LOOK.

Key Terms Associated with Disk Scheduling

- **Seek Time:** Time taken to move the disk arm to the track where data is located.
- **Rotational Latency:** Time taken for the desired sector to rotate under the read/write head.
- **Transfer Time:** Time taken to actually read or write the data, depending on disk speed and data size.

$$\text{Disk Access Time} = \text{Seek Time} + \text{Rotational Latency} + \text{Transfer Time}$$

$$\text{Total Seek Time} = \text{Total head Movement} * \text{Seek Time}$$



Disk Response Time

- **Response Time:** The time a request waits before its I/O operation starts.
- **Average Response Time:** The mean waiting time of all requests.

- **Variance in Response Time:** How much individual waiting times differ from the average.

Goals of Disk Scheduling Algorithms

- Minimize Seek Time
- Maximize Throughput
- Minimize Latency
- Ensuring Fairness
- Efficiency in Resource Utilization

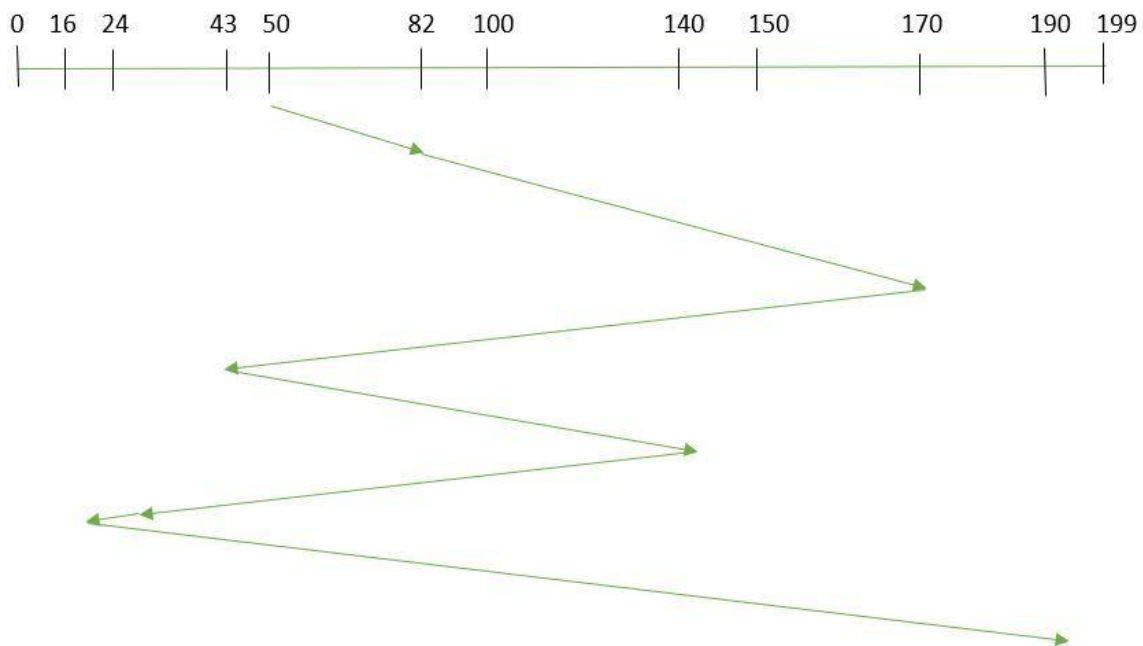
DISK SCHEDULING ALGORITHMS

1. FCFS (First Come, First Served)

- Requests are served **in the order they arrive**.
- No reordering or optimization.

Example

Suppose the order of request is- (82,170,43,140,24,16,190) and current position of Read/Write head is: 50



So, total overhead movement (total distance covered by the disk arm) = $(82-50) + (170-82) + (170-43) + (140-43) + (140-24) + (24-16) + (190-16) = 642$

Example 2:

Sequence:

Head = 53

Requests: 98, 183, 37, 122, 14, 124, 65, 67

Schedule the above using FCFS and calculate total head movement.

Advantages

Simple, fair (no starvation).

Disadvantages

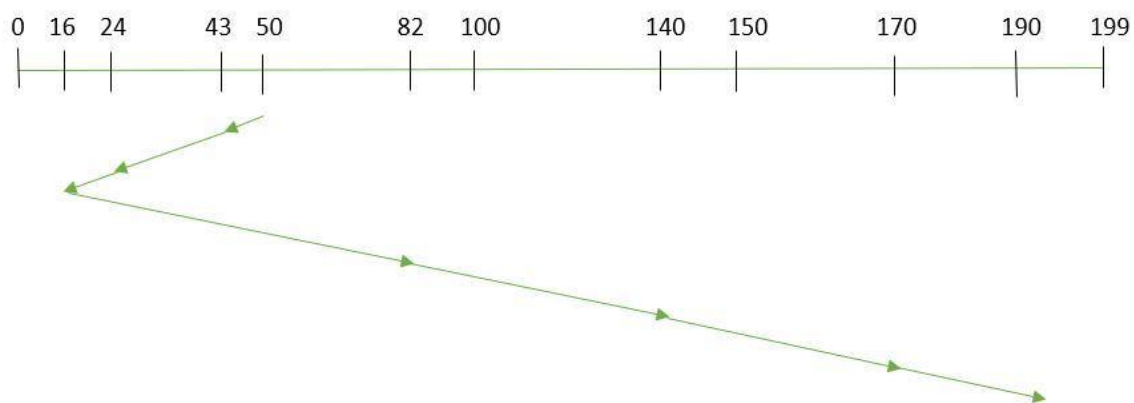
High total head movement → inefficient.

2. SSTF (Shortest Seek Time First)

- Choose the **closest pending request** to the current head position.
- Reduces total head movement compared to FCFS.

Example1:

Suppose the order of request is- (82,170,43,140,24,16,190) and current position of Read/Write head is: 50



total overhead movement (total distance covered by the disk arm) = $(50-43)+(43-24)+(24-16)+(82-16)+(140-82)+(170-140)+(190-170) = 208$

Example 2:

Initial head: 53

Requests: 98, 183, 37, 122, 14, 124, 65, 67

Schedule the above using SSTF and calculate total head movement.

Advantages

Greatly reduces seek time.

Disadvantages

May cause **starvation** — requests far away may wait indefinitely.

3. SCAN (Elevator Algorithm)

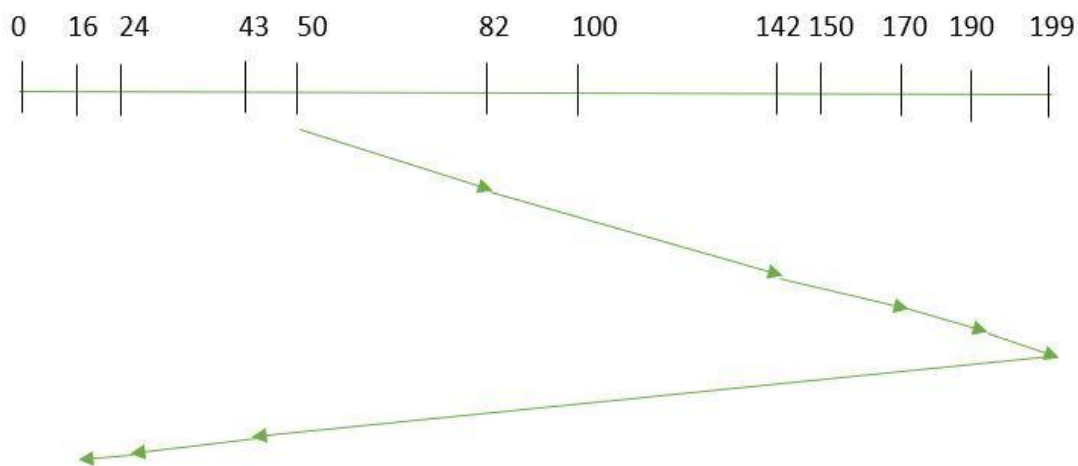
- Disk arm moves like an **elevator**:
 - Moves in one direction (say up) and services requests in order.
 - When it reaches the end, it reverses direction.

Assume:

- Disk range = 0–199
- Head starts at 53
- Moving **toward higher numbers first**

Example 1:

Suppose the requests to be addressed are -82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value".



Therefore, the total overhead movement (total distance covered by the disk arm) is calculated as

$$= (199-50) + (199-16) = 332$$

Example2:

Initial head: 53

Requests: 98, 183, 37, 122, 14, 124, 65, 67

Schedule the above using SCAN and calculate total head movement.

Advantages

- Provides **fair** and **uniform wait times**.
- Suitable for systems with heavy disk load.

Disadvantages

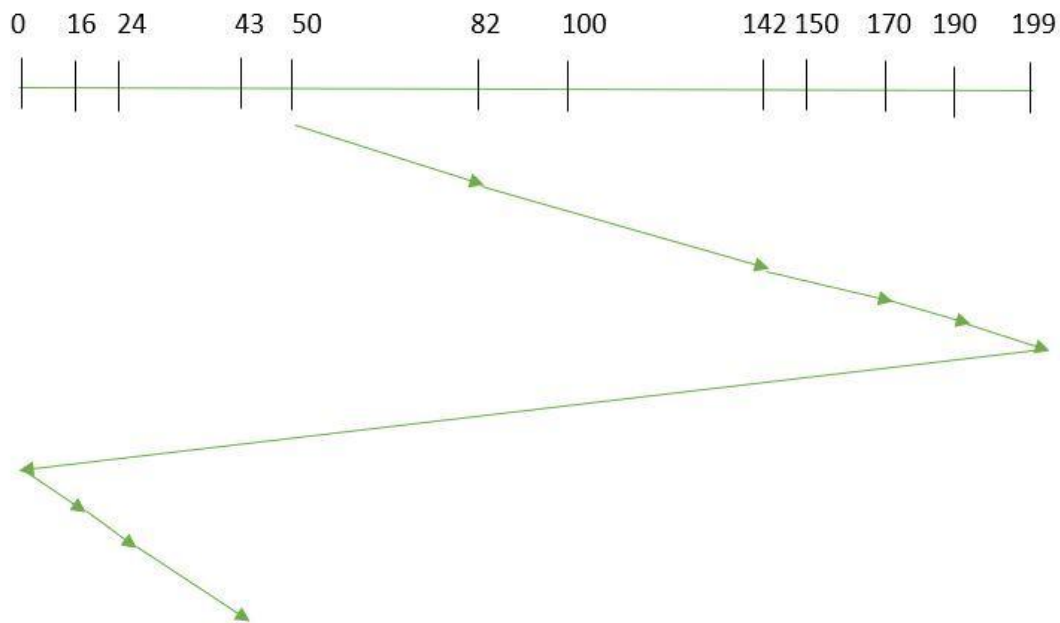
- Edge tracks get **less frequent** service.

4. C-SCAN (Circular SCAN)

- Moves **in one direction only** (say upward).
- After reaching the end, head **returns quickly to the beginning** (without servicing requests).

Example 1:

Suppose the requests to be addressed are- 82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value".



So, the total overhead movement (total distance covered by the disk arm) is calculated as:

$$=(199-50) + (199-0) + (43-0) = 391$$

Example2:

Head = 53

Requests = 14, 37, 65, 67, 98, 122, 124, 183

Schedule the above using C-SCAN and calculate total head movement.

Advantages

- Provides **more uniform response time** (equal waiting for all).
- Avoids bias to inner/outer tracks.

Disadvantages

- Slightly longer total head movement due to jump from end → start.

5. LOOK and C-LOOK

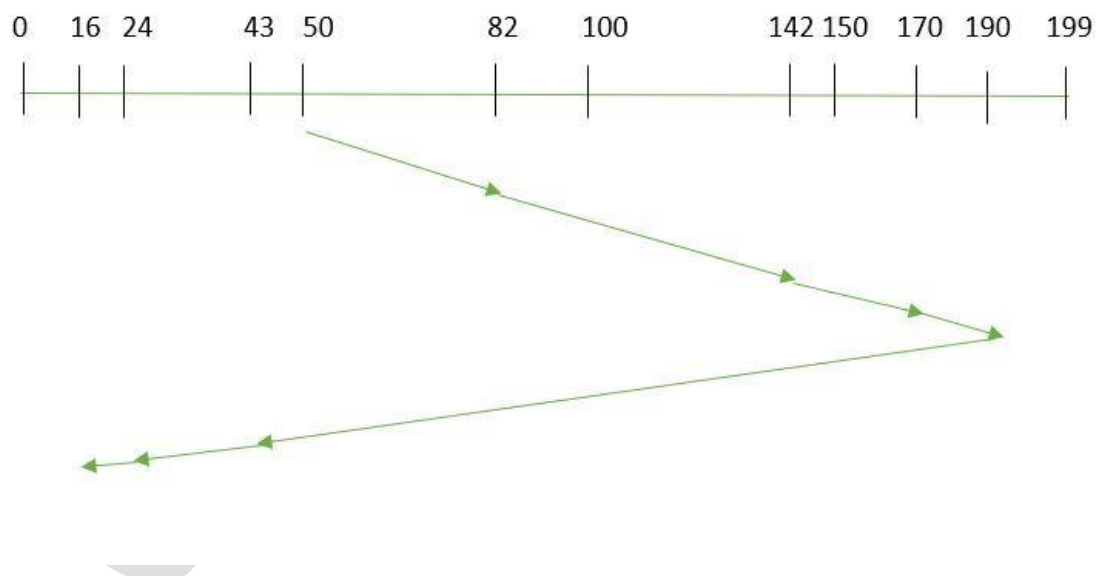
These are **optimized versions** of SCAN and C-SCAN.

LOOK (Modified SCAN)

- Head goes only as far as **last request in that direction**, not till disk end.
- **Better than SCAN** (no movement to extreme end).

Example 1:

Suppose the requests to be addressed are- 82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value".



So, the total overhead movement (total distance covered by the disk arm) is calculated as:

$$= (190-50) + (190-16) = 314$$

Example 2 (direction: upward first)

Initial head: 53

Requests: 14, 37, 65, 67, 98, 122, 124, 183

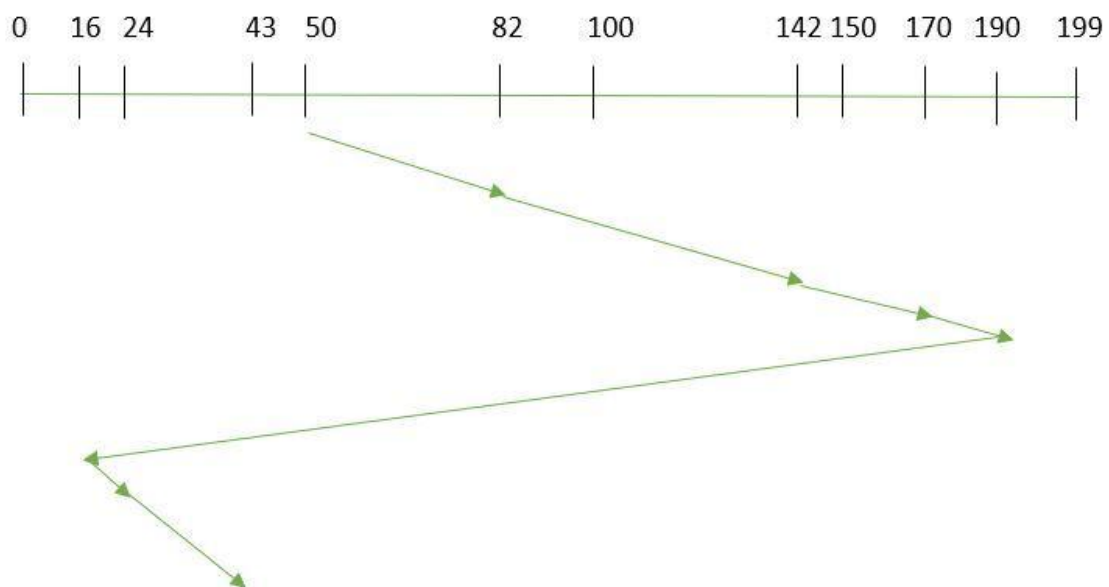
Schedule the above using LOOK and calculate total head movement.

C-LOOK (Modified C-SCAN)

- Like C-SCAN, but head jumps **only to the lowest requested cylinder**, not 0.

Example 1:

Suppose the requests to be addressed are-82,170,43,140,24,16,190 and the Read/Write arm is at 50, and it is also given that the disk arm should move "towards the larger value"



So, the total overhead movement (total distance covered by the disk arm) is calculated as

$$= (190-50) + (190-16) + (43-16) = 341$$

Example 2:**Initial head: 53**

Requests = 14, 37, 65, 67, 98, 122, 124, 183

Schedule the above using C- LOOK and calculate total head movement.

Advantages:

- **Uniform Wait Time:** Requests are serviced in a circular manner, so waiting times are more predictable and fair.
- **Reduced Head Movement:** The arm only goes as far as the last request in one direction, then jumps back, saving time compared to C-SCAN.